

Unit 3—Lesson 2:

Type Casting and Inspection

Type inspection

```
func getClientPet() -> Animal {  
    //returns the pet  
}
```

```
let pet = getClientPet() //`pet` is of type `Animal`
```

Type inspection

```
if pet is Dog {  
    print("The client's pet is a dog")  
} else if pet is Cat {  
    print("The client's pet is a cat")  
} else if pet is Bird {  
    print("The client's pet is a bird")  
} else {  
    print("The client has a very exotic pet")  
}
```


Type inspection

```
let pets = allPets() //`pets` is of type `[Animal]`  
var dogCount = 0, catCount = 0, birdCount = 0  
for pet in pets {  
    if pet is Dog {  
        dogCount += 1  
    } else if pet is Cat {  
        catCount += 1  
    } else if pet is Bird {  
        birdCount += 1  
    }  
}  
print("Brad looks after \$(dogCount) dogs, \$(catCount) cats, and \$(birdCount) birds.")
```

Type casting

- Type casting in Swift is the process of converting an object from one type to another.
- The syntax of type casting in Swift is as follows:

```
expression as Type
```

```
let stringValue = "hello"  
let intValue = 42  
let doubleValue = Double(intValue) // 42.0  
let stringDoubleValue = Double(stringValue) // This will fail and return nil
```

- The first typecast converts an integer to a Double.
- The second typecast tries to convert a string “hello” to a Double. Since “hello” is not a number, the typecast fails and returns nil.

Type Casting

Upcasting and Downcasting:

- Upcasting: Converting a subclass instance to its superclass type. This is safe and implicit.
- Downcasting: Converting a superclass instance to a subclass type. This requires explicit casting and can fail if the instance isn't of the target subclass type.

Type Casting Operators:

- **as?** (Conditional Downcast): Attempts to downcast and returns an optional. If the downcast fails, it returns nil.
- **as!** (Forced Downcast): Forces a downcast and triggers a runtime error if it fails. Use only when you're certain of the instance's type.
- **is** (Type Check): Checks if an instance is of a certain type and returns a Boolean value.

Type Casting Operators

Operator	Purpose	Result Type
is	Check if an instance is of a certain subclass type.	Bool
as?	Conditional cast. Returns nil if the cast fails.	Optional (Type?)
as!	Forced cast. Crashes your app if the cast fails.	Non-optional (Type)

Downcasting vs. Upcasting

- **Downcasting** is used to convert a value from a superclass type to a subclass type. The syntax for downcasting is as follows:

```
let instance = superclassValue as? SubclassType
```

- The **as?** operator is used to perform a downcast and returns an optional value of type SubclassType. If the downcast is successful, the optional will contain the downcasted value, otherwise it'll be nil.
- Upcasting is used to convert a value from a subclass type to a superclass type. The syntax for upcasting is as follows:

```
let instance: SuperclassType = subclassValue as SuperclassType
```

- The **as** operator is used to perform an upcast. If the upcast is successful, the expression will be of type SuperclassType, otherwise it'll cause a compile-time error.

Upcasting

```
class Animal {  
    func makeSound() {  
        print("Some sound")  
    }  
}  
  
class Dog: Animal {  
    func bark() {  
        print("Woof!")  
    }  
}  
  
let dog = Dog()  
let animal: Animal = dog // Upcasting  
animal.makeSound() // Outputs: Some sound
```

Downcasting

Downcasting with as?

```
let animal: Animal = Dog()  
if let dog = animal as? Dog {  
    dog.bark() // Outputs: Woof!  
} else {  
    print("Not a Dog")  
}
```

Downcasting with as!

```
let animal: Animal = Dog()  
let dog = animal as! Dog  
dog.bark() // Outputs: Woof!
```

Using is for Type Checking

```
let animal: Animal = Dog()
if animal is Dog {
    print("This is a Dog")
} else {
    print("This is not a Dog")
}
```

Type casting

```
func walk(dog: Dog) {  
    print("Walking \"(dog.name)\"")  
}  
  
func cleanLitterBox(cat: Cat) { . . . }  
  
func cleanCage(bird: Bird) { . . . }  
  
for pet in pets {  
    if pet is Dog {  
        walk(dog: pet) // Compiler error. The compiler sees `pet` as an `Animal`, not a `Dog`.  
    }  
    . . .  
}
```



Type casting

```
for pet in pets {  
  if let dog = pet as? Dog {  
    walk(dog: dog)  
  } else if let cat = pet as? Cat {  
    cleanLitterBox(cat: cat)  
  } else if let bird = pet as? Bird {  
    cleanCage(bird: bird)  
  }  
}
```

Any

- Homogenous collections are simpler to work with because you know the type of every instance within them.
- But what if you want to work with nonspecific types?
- Swift provides two special types: `Any` and `AnyObject`.
- As the name implies, `Any` can represent an instance of any type: strings, doubles, functions, or whatever.
- `AnyObject` can represent an instance of any class in Swift but not a structure

```
var items: [Any] = [5, "Bill", 6.7, Dog()]
```

Any

```
var items: [Any] = [5, "Bill", 6.7, Dog()]
let firstItem = items[0]

if firstItem is Int {
    print("The first element is an integer")
} else if firstItem is String {
    print("The first element is a string")
} else {
    print("The first element is neither an integer nor a string")
}
```


Any

```
var items: [Any] = [5, "Bill", 6.7, Dog()]
```

```
if let firstItem = items[0] as? Int {  
    print(firstItem + 4)  
}
```

9

Casting with Switch

- In Swift, type casting is done using the `is` and `as` operators. When working with a `switch` statement, you can use type casting to check the type of a value at runtime and perform different actions based on the actual type.
1. `is` **Operator**: Checks if a value is of a specific type.
 2. `as?` **Operator**: Attempts to cast a value to a specific type (returns an optional).
 3. `as!` **Operator**: Forcefully casts a value to a specific type (crashes if the cast fails).

Example

```
class Animal {}
class Dog: Animal {
    func bark() {
        print("Woof! Woof!")
    }
}
class Cat: Animal {
    func meow() {
        print("Meow! Meow!")
    }
}
```

```
let animals: [Animal] = [Dog(), Cat(), Dog()]

for animal in animals {
    switch animal {
    case let dog as Dog: // Cast to Dog
        dog.bark()
    case let cat as Cat: // Cast to Cat
        cat.meow()
    default:
        print("Unknown animal")
    }
}
```

OUTPUT:

Woof! Woof!
Meow! Meow!
Woof! Woof!

NOTE: This is a useful way to handle multiple types in a structured way, avoiding the need for multiple `if let` or `guard let` statements.

Unit 3—Lesson 2

Lab: Type Casting and Inspection



Open and complete the exercises in Lab - Type Casting.playground

